Task 2

Advanced Data Preprocessing and Feature Engineering

Predictive Analytics & AI Model Deployment - Customer Churn Prediction

Objective. Clean the data, handle missing values and outliers, encode/scale, and engineer informative features.

1. Pipeline overview

- **Audit** the raw data (duplicates, missing values, impossible values, inconsistent labels).
- **Clean**: remove 180 duplicates -> 30,000 clean rows.
- **Split** 80/20, stratified on churn: 24,000 train / 6,000 test rows (churn 26.9% in both).
- **Feature engineering**: 9 new features (table below) on top of 16 raw ones.
- **Encode & scale**: median imputation + standardisation for numerics; mode imputation + one-hot for categoricals.
- **No leakage**: all statistics (medians, means, categories) are learned from the training set only. The final feature matrix is 24000 rows x 45 columns with 0 missing values.

2. Data-quality audit

Issue	Found in raw data	Fix applied
Duplicate rows	180	drop_duplicates before splitting
Missing values	age (599), region (332), total_charges (897), avg_monthly_usage_gb (576), satisfaction_score (1209)	Median (numeric) / mode (categorical) imputation inside the pipeline
Out-of-range values	age (95), monthly_charges (177)	Validated against plausible ranges, set to NaN, then imputed
Inconsistent gender text	1510	strip + Title Case normalisation
Inconsistent contract text	1512	strip + Title Case normalisation

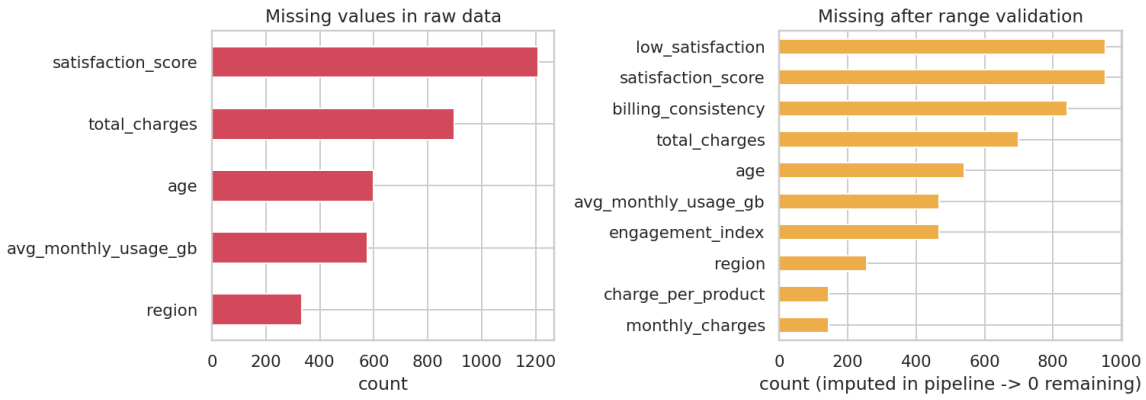


Figure 1 - Missing values before and after range validation

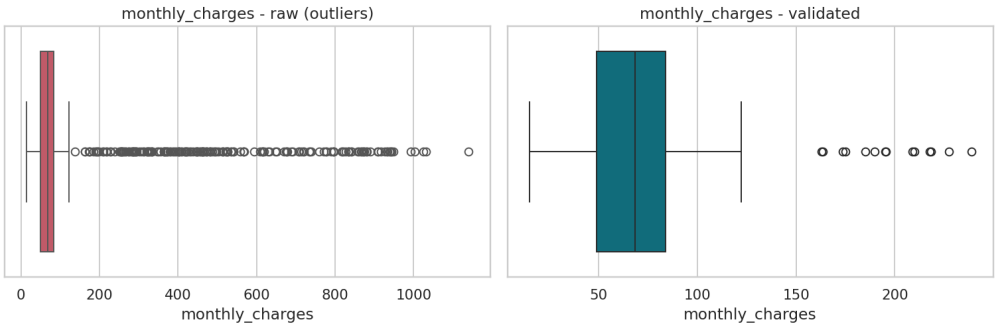


Figure 2 - Outliers in monthly_charges removed by range validation

3. Feature engineering

Engineered feature	Formula	Business meaning
charge_per_product	monthly_charges / num_products	Price paid per product
billing_consistency	total_charges / (monthly x tenure + 1)	Detects billing anomalies / plan changes
ticket_rate	tickets / (tenure + 1) x 12	Support tickets per year of relationship
late_rate	late_payments / (tenure + 1) x 12	Payment-risk intensity
engagement_index	usage_gb / (days_since_login + 1)	Recent activity vs. volume
low_satisfaction	satisfaction <= 4	Flag for unhappy customers
monthly_fiber_flag	Monthly contract AND Fiber	Expensive, uncommitted segment
tenure_group	0-12, 13-24, 25-48, 49+	Non-linear tenure effect
age_group	<=30, 31-45, 46-60, 60+	Age band

4. Exploratory analysis

Churn is strongly linked to contract type, tenure and payment method (Figure 3). The engineered *monthly_fiber_flag* has the largest correlation with churn of any numeric feature (r = 0.25), followed by satisfaction score (-0.23) and tenure (-0.20) - evidence that the engineered features add signal (Figure 4).

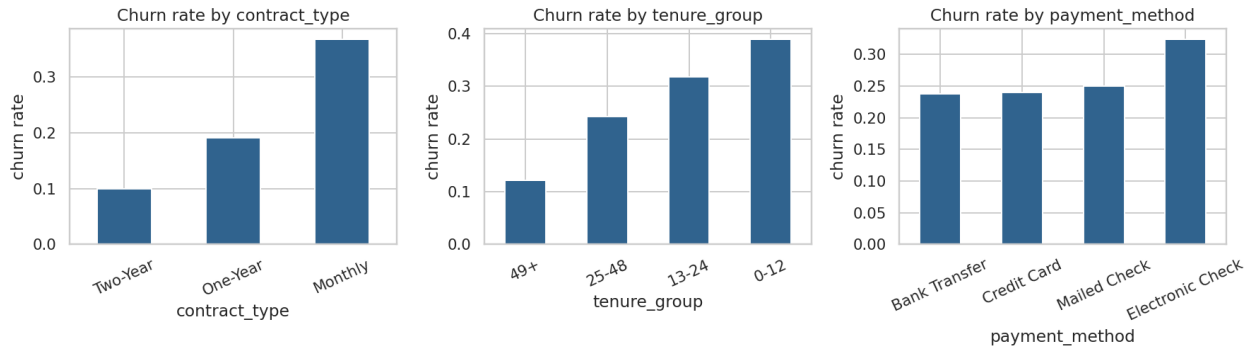


Figure 3 - Churn rate by contract, tenure group and payment method

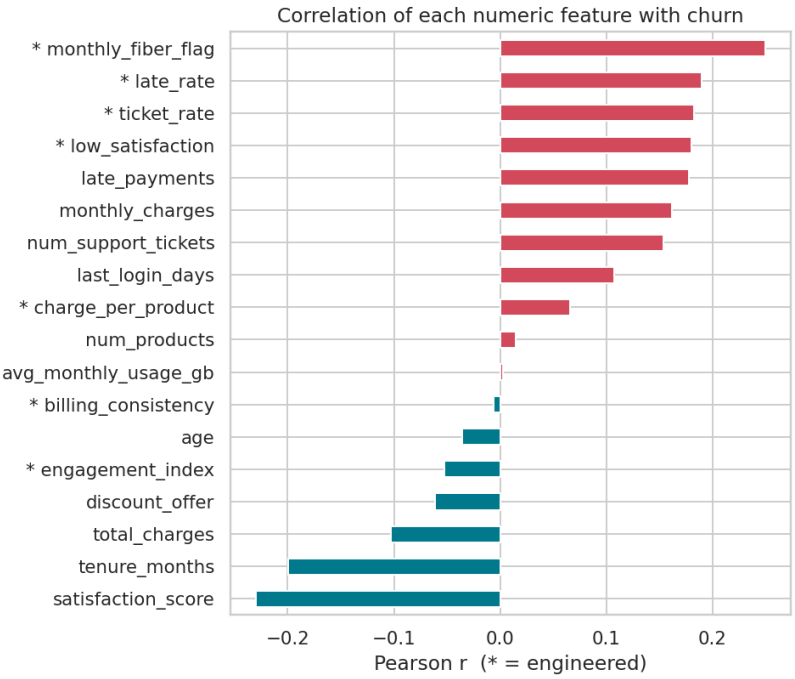


Figure 4 - Correlation with churn (* = engineered feature)

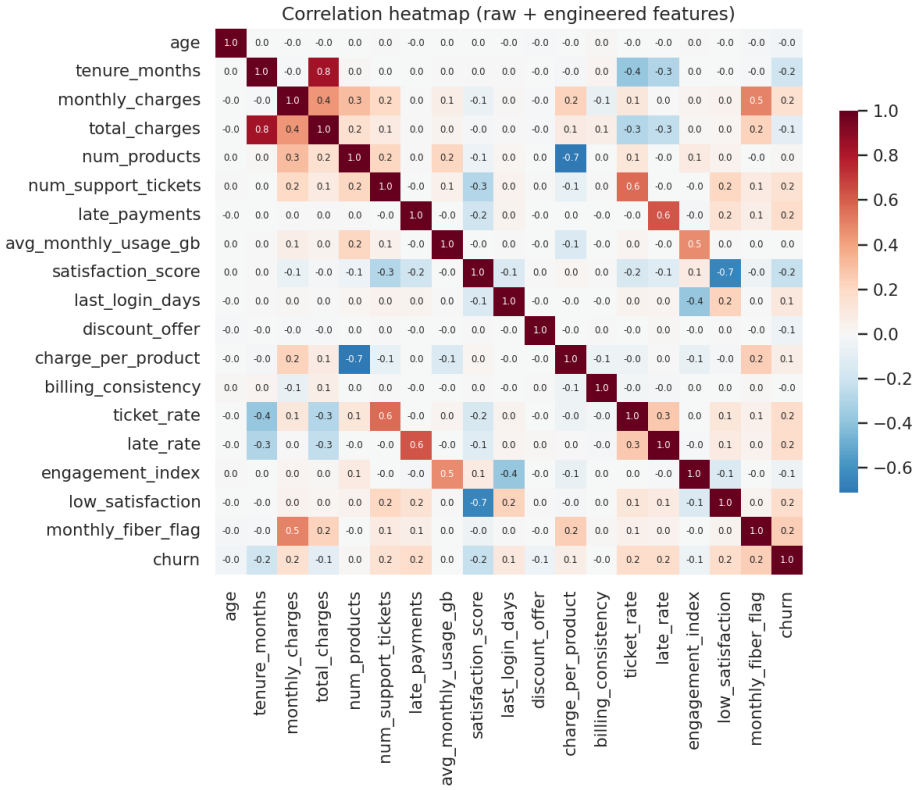


Figure 5 - Correlation matrix of numeric features

5. Output

- `data/processed/train.csv` and `test.csv`.
- `src/features.py` - the reusable cleaning + feature pipeline used by training and the API.

Appendix - Source code

src/features.py

```
"""Cleaning rules, feature engineering and the sklearn preprocessing pipeline.

The full pipeline (feature engineering -> imputation/scaling/encoding -> model)
is a single sklearn Pipeline, so exactly the same steps run in training,
cross-validation and inside the deployed API (no train/serve skew, no leakage).
"""
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import FunctionTransformer, OneHotEncoder, StandardScaler

RAW_NUMERIC = ["age", "tenure_months", "monthly_charges", "total_charges", "num_products",
               "num_support_tickets", "late_payments", "avg_monthly_usage_gb",
               "satisfaction_score", "last_login_days", "discount_offer"]
RAW_CATEGORICAL = ["gender", "region", "contract_type", "internet_service", "payment_method"]
ENG_NUMERIC = ["charge_per_product", "billing_consistency", "ticket_rate", "late_rate",
               "engagement_index", "low_satisfaction", "monthly_fiber_flag"]
ENG_CATEGORICAL = ["tenure_group", "age_group"]
NUMERIC = RAW_NUMERIC + ENG_NUMERIC
CATEGORICAL = RAW_CATEGORICAL + ENG_CATEGORICAL

# Physically plausible ranges: anything outside is treated as a data error -> NaN
VALID_RANGES = {
    "age": (18, 100), "tenure_months": (0, 120), "monthly_charges": (0, 250),
    "total_charges": (0, 30000), "num_products": (1, 10), "num_support_tickets": (0, 50),
    "late_payments": (0, 50), "avg_monthly_usage_gb": (0, 5000), "satisfaction_score": (1, 10),
    "last_login_days": (0, 365), "discount_offer": (0, 1),
}


def add_features(df: pd.DataFrame) -> pd.DataFrame:
    """Standardise raw columns, null out invalid values and engineer new features."""
    df = df.copy()
    for c in RAW_CATEGORICAL:
        if c not in df:
            df[c] = np.nan
        s = df[c].astype(object)
        df[c] = s.where(s.isna(), s.astype(str).str.strip().str.title())
        df[c] = df[c].where(df[c].notna(), np.nan)
    for c in RAW_NUMERIC:
        if c not in df:
            df[c] = np.nan
        df[c] = pd.to_numeric(df[c], errors="coerce")
        lo, hi = VALID_RANGES[c]
        df.loc[(df[c] < lo) | (df[c] > hi), c] = np.nan

    df["charge_per_product"] = df["monthly_charges"] / df["num_products"].clip(lower=1)
    df["billing_consistency"] = df["total_charges"] / (df["monthly_charges"] * df["tenure_months"] + 1)
    df["ticket_rate"] = df["num_support_tickets"] / (df["tenure_months"] + 1) * 12
    df["late_rate"] = df["late_payments"] / (df["tenure_months"] + 1) * 12
    df["engagement_index"] = df["avg_monthly_usage_gb"] / (df["last_login_days"] + 1)
    df["low_satisfaction"] = (df["satisfaction_score"] <= 4).astype(float).where(
        df["satisfaction_score"].notna(), np.nan)
    df["monthly_fiber_flag"] = ((df["contract_type"] == "Monthly") &
                                (df["internet_service"] == "Fiber")).astype(float)
    tg = pd.cut(df["tenure_months"], [-1, 12, 24, 48, 1000], labels=["0-12", "13-24", "25-48", "49+"])
    df["tenure_group"] = tg.astype(object)
    ag = pd.cut(df["age"], [0, 30, 45, 60, 200], labels=["<=30", "31-45", "46-60", "60+"])
    df["age_group"] = ag.astype(object)
    for c in ENG_CATEGORICAL:
        df[c] = df[c].where(df[c].notna(), "Unknown")
    return df[NUMERIC + CATEGORICAL]


def build_preprocessor() -> ColumnTransformer:
    num = Pipeline([("impute", SimpleImputer(strategy="median")), ("scale", StandardScaler())])
    cat = Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                    ("onehot", OneHotEncoder(handle_unknown="ignore", sparse_output=False))])
    return ColumnTransformer([("num", num, NUMERIC), ("cat", cat, CATEGORICAL)])


def make_pipeline(estimator) -> Pipeline:
    """Full end-to-end pipeline: features -> preprocessing -> estimator."""
    return Pipeline([
        ("features", FunctionTransformer(add_features, validate=False)),
        ("prep", build_preprocessor()),
        ("model", estimator),
    ])
```

src/preprocess.py

```
"""TASK 2 - Advanced data preprocessing and feature engineering.

Steps: data-quality audit -> de-duplication -> stratified train/test split ->
fit the preprocessing pipeline on TRAIN only -> EDA figures + summary.
```

```
"""
import json
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from .config import DATA_PROC, FIGS, RAW_FILE, RESULTS, SEED, TARGET
from .features import (CATEGORICAL, NUMERIC, RAW_NUMERIC, VALID_RANGES, add_features,
                       build_preprocessor)

sns.set_theme(style="whitegrid")


def audit(df):
    invalid = {}
    for c, (lo, hi) in VALID_RANGES.items():
        n = int(((df[c] < lo) | (df[c] > hi)).sum())
        if n:
            invalid[c] = n
    return {"rows": len(df), "duplicates": int(df.duplicated().sum()),
            "missing": {k: int(v) for k, v in df.isna().sum().items() if v},
            "out_of_range": invalid,
            "dirty_gender_values": int((df["gender"].dropna() != df["gender"].dropna().str.title()).sum()),
            "dirty_contract_values": int((df["contract_type"].dropna() !=
                                          df["contract_type"].dropna().str.strip().str.title()).sum())}


def main():
    raw = pd.read_csv(RAW_FILE)
    before = audit(raw)
    df = raw.drop_duplicates().dropna(subset=[TARGET]).reset_index(drop=True)
    train, test = train_test_split(df, test_size=0.2, stratify=df[TARGET], random_state=SEED)
    train.to_csv(DATA_PROC / "train.csv", index=False)
    test.to_csv(DATA_PROC / "test.csv", index=False)

    fe_train = add_features(train)
    after = {"missing": {k: int(v) for k, v in fe_train.isna().sum().items() if v}}
    prep = build_preprocessor().fit(fe_train)  # fit on train only (no leakage)
    Xt = prep.transform(fe_train)
    names = list(prep.get_feature_names_out())
    final_missing = int(np.isnan(Xt).sum())

    # ---------- figures ----------
    fig, ax = plt.subplots(1, 2, figsize=(11, 4))
    m1 = pd.Series(before["missing"]).sort_values()
    m1.plot.barh(ax=ax[0], color="#d1495b"); ax[0].set_title("Missing values in raw data")
    ax[0].set_xlabel("count")
    cleaned = pd.Series(after["missing"]).sort_values()
    cleaned.plot.barh(ax=ax[1], color="#edae49"); ax[1].set_title("Missing after range validation")
    ax[1].set_xlabel("count (imputed in pipeline -> 0 remaining)")
    plt.tight_layout(); plt.savefig(FIGS / "t2_missing_values.png", dpi=140); plt.close()

    fig, ax = plt.subplots(1, 2, figsize=(11, 3.8))
    sns.boxplot(x=raw["monthly_charges"], ax=ax[0], color="#d1495b"); ax[0].set_title("monthly_charges - raw (outliers)")
    sns.boxplot(x=fe_train["monthly_charges"], ax=ax[1], color="#00798c"); ax[1].set_title("monthly_charges - validated")
    plt.tight_layout(); plt.savefig(FIGS / "t2_outliers.png", dpi=140); plt.close()

    tmp = fe_train.copy(); tmp[TARGET] = train[TARGET].values
    corr = tmp[NUMERIC + [TARGET]].corr()
    plt.figure(figsize=(9, 7.5))
    sns.heatmap(corr, cmap="RdBu_r", center=0, annot=True, fmt=".1f", annot_kws={"size": 6}, cbar_kws={"shrink": .7})
    plt.title("Correlation heatmap (raw + engineered features)")
    plt.tight_layout(); plt.savefig(FIGS / "t2_correlation.png", dpi=140); plt.close()
    c = corr[TARGET].drop(TARGET).sort_values()
    plt.figure(figsize=(7, 6))
    colors = ["#00798c" if v < 0 else "#d1495b" for v in c]
    c.plot.barh(color=colors); plt.title("Correlation of each numeric feature with churn")
    eng = set(["charge_per_product", "billing_consistency", "ticket_rate", "late_rate",
               "engagement_index", "low_satisfaction", "monthly_fiber_flag"])
    plt.gca().set_yticklabels([("* " if i in eng else "") + i for i in c.index])
    plt.xlabel("Pearson r  (* = engineered)"); plt.tight_layout()
    plt.savefig(FIGS / "t2_target_corr.png", dpi=140); plt.close()

    fig, ax = plt.subplots(1, 3, figsize=(13, 3.8))
    for a, col in zip(ax, ["contract_type", "tenure_group", "payment_method"]):
        tmp.groupby(col)[TARGET].mean().sort_values().plot.bar(ax=a, color="#30638e")
        a.set_title(f"Churn rate by {col}"); a.set_ylabel("churn rate"); a.tick_params(axis="x", rotation=25)
    plt.tight_layout(); plt.savefig(FIGS / "t2_churn_by_segment.png", dpi=140); plt.close()

    summary = {"raw_rows": before["rows"], "clean_rows": len(df), "train_rows": len(train),
               "test_rows": len(test), "audit_before": before, "missing_after_validation": after["missing"],
               "final_matrix_shape": list(Xt.shape), "final_missing": final_missing,
               "n_numeric_raw": len(RAW_NUMERIC), "n_numeric_total": len(NUMERIC),
               "n_categorical": len(CATEGORICAL), "encoded_feature_count": len(names),
               "churn_rate_train": round(float(train[TARGET].mean()), 4),
               "churn_rate_test": round(float(test[TARGET].mean()), 4),
               "top_corr": {k: round(float(v), 3) for k, v in
           c.abs().sort_values(ascending=False).head(8).index.to_series().map(lambda k: c[k]).items()}}
    (RESULTS / "task2_preprocess.json").write_text(json.dumps(summary, indent=2))
    print(json.dumps(summary, indent=2))
if __name__ == "__main__":
    main()
```